

AhorraGo Master Report

Historial acumulado de fases técnicas del proyecto.

Fase 1

Fecha: 2026-05-31

Objetivo

Corregir problemas criticos y de alto impacto inicial sin romper el MVP actual, evitando refactor masivo,

Cambios Aplicados

1. Seguridad: API Key de Jumbo

- Se elimino la API key hardcodeada de los scrapers de Jumbo.
- Los scrapers ahora leen JUMBO_API_KEY desde variables de entorno.
- Si falta la variable, el scraper falla con un mensaje claro sin exponer secretos.
- Se creo .env.example con la variable requerida.
- Se actualizo .gitignore para excluir .env y .env., manteniendo .env.example versionable.

Archivos:

- app/scrapper_jumbo_real.py
- app/scrapper_jumbo_api.py
- .env.example
- .gitignore

2. Busquedas con LIMIT y OFFSET

- Se agregaron parametros limit y offset a endpoints de busqueda.
- Default: limit=50.
- Maximo permitido: limit=100.
- Se evito carga ilimitada en las busquedas principales de productos.
- El frontend ahora consulta con limit=50&offset=0.

Archivos:

- app/main.py
- app/services.py
- frontend/index.html

3. URLs Correctamente Escapadas

- Se creo helper centralizado para generar URLs de busqueda con urllib.parse.urlencode.
- Se reemplazaron URLs generadas con replace(" ", "%20").
- Las URLs ahora soportan tildes, n y caracteres especiales.

Archivos:

- app/url_utils.py
- app/convertir_lider.py
- app/convertir_jumbo.py
- app/convertir_unimarc.py
- app/generar_catalogo.py
- app/services.py

4. Indices Basicos en SQLite

- Se reforzo el script idempotente de indices.
- Se agregaron indices seguros para:
 - productos.nombre
 - productos.producto_base
 - precios.producto_id
 - precios.supermercado_id
- El script usa CREATE INDEX IF NOT EXISTS, por lo que no borra ni reconstruye datos.

Archivo:

- app/scripts/agregar_indices.py

5. Lock Real para Actualizacion de Productos

- Se reemplazo el lock manual por filelock.
- Se evita que dos ejecuciones del pipeline corran al mismo tiempo.
- Si ya existe una actualizacion en curso, el proceso sale con mensaje claro.

Archivos:

- app/actualizar_productos.py
- requirements.txt

6. Tests Iniciales

Se creo el primer set minimo de tests para:

- Normalizacion de texto.
- Equivalencia entre Leche Soprole 1L y Leche Soprole 1000ml.
- URLs con tildes y n.
- Limites de busqueda.
- Calculo basico de ahorro.

Archivo:

- tests/test_phase1.py

7. Documento para Agentes

Se creo AGENTS.md con:

- Stack detectado.
- Reglas de trabajo para Codex.
- Comandos de ejecucion.
- Comandos de testing.
- Reglas de seguridad.

- Definicion de terminado.

Archivo:

- AGENTS.md

Dependencias Agregadas

En requirements.txt:

```
"""txt
filelock>=3.13.0
pytest>=8.0.0
"""
```

Verificacion Ejecutada

Comandos ejecutados:

```
""powershell
venv\Scripts\python.exe -m pytest tests/test_phase1.py -q
venv\Scripts\python.exe -m compileall app tests
venv\Scripts\python.exe -m app.scripts.agregar_indices
venv\Scripts\python.exe -c "from app.main import app; print(app.title)"
"""
```

Resultados:

- Tests: 5 passed.
- Compilacion Python: OK.
- Import de FastAPI app: OK.
- Indices SQLite aplicados correctamente.
- Verificacion de JUMBO_API_KEY faltante: muestra error claro.

Archivos Modificados o Creados

Modificados:

- .gitignore
- app/actualizar_productos.py
- app/convertir_jumbo.py
- app/convertir_lider.py
- app/convertir_unimarc.py
- app/generar_catalogo.py
- app/importar_csv.py
- app/main.py
- app/scrapper_jumbo_api.py
- app/scrapper_jumbo_real.py
- app/scripts/agregar_indices.py
- app/services.py
- frontend/index.html
- requirements.txt

Creados:

- .env.example
- AGENTS.md
- app/url_utils.py
- tests/test_phase1.py
- CAMBIOS_FASE_1.md

Riesgos Pendientes

- Aun existen .all() en endpoints de diagnostico, estado de datos y logica interna no critica. No se toca
- El pipeline completo de scrapers no fue ejecutado para evitar scraping externo y cambios masivos de dat
- Hay un warning de Pydantic por uso de @validator; conviene migrarlo a @field_validator en una fase post
- SQLite se mantiene como base actual, segun la restriccion de no migrar todavia.

Siguiente Fase Recomendada

1. Optimizar endpoints pesados como /diagnostico/calidad y /estado-datos.
2. Agregar tests de integracion con SQLite temporal.
3. Migrar validadores Pydantic a estilo v2.
4. Revisar limites y paginacion en comparador y diagnosticos.
5. Revisar manejo de configuracion con carga explicita de .env si el flujo local lo necesita.

Fase 2

Fecha: 2026-05-31

Objetivo

Reducir deuda tecnica sin romper el MVP, enfocandose en main.py, normalizacion duplicada, matching de pro

Cambios Aplicados

1. Normalizacion compartida

- Se creo app/normalizacion.py.
- Se movieron funciones de normalizacion y comparacion de claves desde main.py.
- importar_csv.py ahora usa generar_producto_base desde el modulo compartido.
- Se mantuvo compatibilidad con imports existentes usados por tests.
- Se normalizaron equivalencias de formato:
 - 1L y 1000ml.
 - 500g y 0.5kg.
 - pack 6 y 6 unidades.

Archivos:

- app/normalizacion.py
- app/main.py
- app/importar_csv.py

2. Matching de productos

- Se creo app/matching.py.
- Se movio candidato_compatible y sus reglas relacionadas a un modulo dedicado.
- main.py ahora importa esa logica en vez de contenerla directamente.
- Se agregaron tests unitarios de equivalencias.

Archivos:

- app/matching.py
- app/main.py
- tests/test_matching.py

3. Reduccion segura de main.py

- Se retiro logica de normalizacion y matching desde main.py.
- Los endpoints actuales se mantienen con las mismas rutas y estructura de respuesta.
- No se reestructuro la API ni se cambio la UX.

Archivo:

- app/main.py

4. Optimizacion de endpoints pesados

- /diagnostico/calidad ahora consulta columnas necesarias y usa yield_per(500) en vez de cargar objetos c
- /estado-datos ahora calcula conteos por supermercado con GROUP BY y conteos directos.
- Se mantuvo la forma de respuesta esperada por el frontend.

Archivo:

- app/main.py

5. Tests de integracion minimos

- Se agrego SQLite temporal en memoria para pruebas de API.
- Se prueba busqueda basica con datos minimos.
- Se prueba limite de resultados.
- Se prueba estado de datos con conteos basicos.

Archivo:

- tests/test_integration.py

6. Pydantic v2

- Se migro @validator a @field_validator.
- El warning de Pydantic v2 desaparece en la suite de tests.

Archivo:

- app/schemas.py

7. Dependencias

Se agrego httpx, requerido por fastapi.testclient.TestClient para los tests de integracion.

Archivo:

- requirements.txt

Comandos Ejecutados

```
""powershell
$env:PATH = "$PWD\env\Scripts;$env:PATH"; python -m pytest -q
$env:PATH = "$PWD\env\Scripts;$env:PATH"; python -m compileall app tests
$env:PATH = "$PWD\env\Scripts;$env:PATH"; python -c "from app.main import app; print(app.title)"
""
```

Resultados

- Tests: 11 passed.
- Compilacion: OK.
- Import de FastAPI app: OK, salida FastAPI.

Archivos Modificados o Creados

Modificados:

- app/importar_csv.py
- app/main.py
- app/schemas.py
- requirements.txt

Creados:

- app/normalizacion.py
- app/matching.py
- tests/test_matching.py
- tests/test_integration.py
- CAMBIOS_FASE_2.md

Riesgos Pendientes

- Todavia quedan .all() en rutas livianas (categorias, subcategorias) y en logica interna acotada; no se
- main.py aun contiene logica de busqueda y resumen de compra que podria separarse en una fase posterior.
- El matching sigue siendo heuristico; requiere mas fixtures reales para cubrir casos de marcas, sabores,
- No se ejecuto pipeline completo de scrapers para evitar cambios masivos de datos.

Siguiente Fase Recomendada

1. Extraer busqueda de productos desde main.py a un service dedicado.
2. Agregar fixtures realistas para matching por supermercado.
3. Cubrir /productos/resumen-compra con tests de integracion.
4. Revisar endpoints internos con .all() restantes y paginar si empiezan a crecer.
5. Evaluar carga explicita de .env local para mejorar experiencia del scraper sin exponer secretos.

Fase 3

Fecha: 2026-05-31

Objetivo

Aumentar la precision del matching de productos y del comparador sin cambiar la arquitectura principal, s

Cambios Aplicados

1. Normalizacion avanzada

Se amplio app/normalizacion.py para reconocer equivalencias de formato:

- Volumen:
 - 1L = 1000ml
 - 1.5L = 1500ml
 - 2L = 2000ml
- Peso:
 - 1kg = 1000g
 - 0.5kg = 500g
 - 250g = 0.25kg
- Cantidad:
 - pack 6
 - 6 un
 - 6 unidades
 - x6

Esto reduce falsos negativos cuando distintos supermercados publican el mismo formato con unidades difere

2. Extractor de atributos estructurados

Se agrego extraer_atributos(nombre_producto), que devuelve senales estructuradas para el matching:

```
"""python
{
  "marca": "soprole",
  "volumen": 1000,
  "peso": None,
  "unidad": "ml",
  "cantidad": None,
  "categoria": "leche",
  "sabores": [],
  "tokens": [...]
}
"""
```

Ejemplo cubierto por tests:

```
"""python
extraer_atributos("Leche Soprole Entera 1L")
"""
```

Devuelve marca soprole, volumen 1000, unidad ml y categoria leche.

3. Matching Score 0-100

Se creo `matching_score(producto, candidato)` en `app/matching.py`.

El score combina:

- Marca.
- Volumen o peso normalizado.
- Cantidad o pack.
- Categoria/familia.
- `producto_base`.
- Interseccion de palabras clave.
- Similitud textual con RapidFuzz.

`candidato_compatible` ahora mantiene filtros duros para evitar falsos positivos y usa el score como senal

4. RapidFuzz

Se agrego `rapidfuzz` a `requirements.txt` y se usa `fuzz.token_set_ratio` para evitar depender de comparacione

Esto mejora casos como:

- Coca Cola vs Coca-Cola.
- Reordenamientos de palabras.
- Diferencias menores en nombres publicados por supermercado.

5. Casos problematicos cubiertos

Se agregaron tests para:

- Leche Soprole 1L vs Leche Soprole 1000ml.
- Coca Cola 1.5L vs Coca Cola 1500ml.
- Arroz Tucapel 1kg vs Arroz Tucapel 1000g.
- Pack 6 Coca Cola vs Coca Cola x6.
- Yogurt de sabores distintos, que no debe coincidir.

6. Fixtures reales

Se creo `tests/fixtures/productos_reales.json` con ejemplos representativos de Jumbo, Lider y Unimarc.

Incluye pares equivalentes y un caso negativo de yogurt con sabores distintos.

Archivos Modificados o Creados

Modificados:

- `app/normalizacion.py`
- `app/matching.py`
- `requirements.txt`

Creados:

- tests/test_matching_score.py
- tests/test_fixtures_matching.py
- tests/fixtures/productos_reales.json
- CAMBIOS_FASE_3.md

Tests Agregados

- Tests unitarios para extractor de atributos.
- Tests unitarios para score de matching.
- Tests con fixtures reales.
- Tests negativos para evitar coincidencias incorrectas por sabor.

Comandos Ejecutados

```
""powershell
$env:PATH = "$PWD\env\Scripts;$env:PATH"; python -m pytest -q
$env:PATH = "$PWD\env\Scripts;$env:PATH"; python -m compileall app tests
$env:PATH = "$PWD\env\Scripts;$env:PATH"; python -c "from app.main import app; print(app.title)"
""
```

Resultados

- Tests: 19 passed.
- Compilacion: OK.
- Import de FastAPI app: OK, salida FastAPI.

Precision Mejorada

La precision mejora porque el matching ya no depende solo de texto o formato literal:

- Las unidades se comparan en una representacion comun.
- Los packs se detectan como cantidad.
- La marca se normaliza antes de comparar.
- RapidFuzz tolera diferencias de orden, guiones y texto adicional.
- Los sabores conocidos actuan como freno para evitar falsos positivos.

Riesgos Pendientes

- El sistema de sabores aun es una lista curada y puede necesitar ampliacion con datos reales.
- El umbral de matching_score ≥ 68 esta calibrado con tests iniciales, pero debe validarse con mas produ
- Algunas categorias complejas, como detergentes, papel higienico y promociones multipack, podrian requer
- No se ejecuto el pipeline completo de scrapers para evitar cambios masivos de datos.

Proximos Pasos Recomendados

1. Medir precision con una muestra real etiquetada manualmente.
2. Agregar fixtures para detergentes, papel higienico, aceites y cafes.
3. Registrar matching_score en diagnosticos internos para revisar falsos positivos.
4. Ajustar umbrales por categoria si aparecen patrones distintos.

5. Crear un reporte de productos agrupados por producto_base para detectar errores de equivalencia.

Fase 4

Fecha: 2026-05-31

Objetivos

Mejorar la calidad de datos y aumentar equivalencias reales entre supermercados antes de crear usuarios,

- Sin usuarios.
- Sin login.
- Sin cambios de frontend.
- Sin migracion de base de datos.
- Sin scraping masivo.
- Sin modificar datos reales.
- Con respaldo previo.
- Con simulacion por defecto.

Metricas Iniciales

- Productos: 31.124
- Precios: 31.139
- Producto_base conflictivos: 1.601
- Productos duplicados: 244
- Productos sospechosos: 3
- Equivalencias detectadas: 2.538
- Sin equivalencia: 79,94%

Metricas Finales

La fase fue de diagnostico y simulacion, por lo que no cambio la base real.

- Total productos analizados: 31.124
- Producto_base unicos: 27.418
- Grupos con equivalencia: 2.538
- Porcentaje sin equivalencia recalculado por diagnostico: 81,03%
- Producto_base conflictivos clasificados: 1.601
- Score promedio muestreado: 70,91
- Productos evaluados en simulacion: 31.124
- Producto_base que cambiarian en simulacion: 29.542
- Porcentaje de cambio simulado: 94,92%
- Grupos con equivalencia propuestos: 915
- Riesgo bajo: 2.001 productos
- Riesgo medio: 126 productos
- Riesgo alto de falso positivo: 2 productos

Conflictos Encontrados

Clasificaciones mas frecuentes:

- Formato y peso distinto: 856 grupos.
- Formato y volumen distinto: 659 grupos.
- Marca distinta: 21 grupos.
- Error de normalizacion con volumen distinto: 21 grupos.
- Sabor distinto con volumen distinto: 7 grupos.

Categorias con Peor Matching

- Frutas y Verduras: 7,41% equivalencia.
- Congelados: 7,70% equivalencia.
- Panaderia: 13,12% equivalencia.
- Desayuno y Snacks: 14,24% equivalencia.
- Carnes y Pescados: 14,90% equivalencia.

Categorias con Mejor Matching

- Mascotas: 57,18% equivalencia.
- Bebidas: 28,01% equivalencia.
- Limpieza: 27,50% equivalencia.
- Bebe: 24,47% equivalencia.
- Higiene Personal: 23,35% equivalencia.

Archivos Creados

- app/matching_diagnostics.py
- app/scripts/diagnostico_matching.py
- app/scripts/simular_reconstruccion_producto_base.py
- tests/test_fase4_diagnostics.py
- reports/diagnostico_matching.md
- reports/conflictos_clasificados.csv
- reports/equivalencias_por_categoria.csv
- reports/grupos_sospechosos.csv
- reports/simulacion_producto_base.csv
- reports/simulacion_producto_base.md
- reports/FASE_4_REPORTE.md
- reports/FASE_4_REPORTE.pdf
- reports/AHORRAGO_MASTER_REPORT.pdf
- CAMBIOS_FASE_4.md

Archivos Modificados

- app/main.py
- app/normalizacion.py
- tests/fixtures/productos_reales.json
- tests/test_integration.py

Scripts Creados

- python -m app.scripts.diagnostics_matching
- python -m app.scripts.simular_reconstruccion_producto_base

Tests Agregados

- Clasificación de conflictos.
- Métricas por categoría.
- Simulación de reconstrucción sin modificar la base.
- Nuevas reglas de normalización.
- Falsos positivos y falsos negativos.
- Endpoint /diagnostico/matching.

Seguridad de Datos

Se creó respaldo automático antes de la fase:

- backups/supercheck_20260531_213244.db

La simulación no modificó datos reales.

Riesgos Pendientes

- El 94,92% de cambios propuestos en simulación indica que no debe aplicarse automáticamente.
- La simulación propone menos grupos con equivalencia que el estado actual; sirve para diagnóstico, no para
- El mayor problema actual está en grupos producto_base demasiado amplios con formatos/pesos/volumenes in
- Se requiere revisión manual o reglas por categoría antes de cualquier limpieza real.

Recomendaciones

1. No aplicar reconstrucción automática todavía.
2. Priorizar corrección de producto_base en bebidas, limpieza, mascotas y lácteos.
3. Crear una muestra etiquetada manualmente de 300 a 500 productos.
4. Ajustar reglas por categoría con métricas de precisión y recall.
5. Mantener /diagnostico/matching como endpoint interno antes de cambios de datos.

Fase 5A

Fecha: 2026-05-31

Objetivo

Mejorar equivalencias únicamente en categorías de mejor retorno y menor riesgo:

- Bebidas
- Limpieza
- Higiene Personal
- Bebe
- Mascotas

No se modificaron categorías de alto riesgo como Frutas y Verduras, Carnes y Pescados, Panadería o Congel

Modo de Trabajo

La fase se ejecuto completa en DRY RUN:

- No se modifiko la base real.
- No se ejecuto scraping.
- No se modifiko frontend.
- No se crearon usuarios.

Reglas Implementadas

Bebidas

- Normalizacion de 1L, 1000ml, 1.5L, 1500ml, 2L, 2000ml.
- Packs pack 6, x6, pack 12, x12.
- Diferenciacion de retornable/no retornable.
- Diferenciacion de zero/sin azucar/light vs normal.
- Proteccion contra sabores distintos.

Limpieza

- Normalizacion de ml, litros, gramos y kilogramos.
- Deteccion de aroma.
- Deteccion de concentrado, diluido, antibacterial y tradicional.
- Proteccion contra Lavanda vs Limon y Antibacterial vs Tradicional.

Higiene Personal

- Deteccion de shampoo, acondicionador, jabon, desodorante, pasta dental y crema.
- Deteccion de aroma.
- Deteccion de genero/formato: hombre, mujer, infantil, adulto.
- Proteccion contra hombre vs mujer e infantil vs adulto.

Bebe

- Deteccion de etapa.
- Deteccion de talla.
- Deteccion de cantidad.
- Proteccion contra Talla M vs Talla G y Etapa 1 vs Etapa 2.

Mascotas

- Deteccion de perro/gato.
- Deteccion de peso.
- Deteccion de etapa: cachorro, adulto, senior.
- Proteccion contra cachorro vs adulto y gato vs perro.

Metricas Actuales vs Proyectadas

- Productos evaluados: 10.656
- Productos equivalentes actuales: 2.918
- Productos equivalentes proyectados: 4.477

- Mejora proyectada: 53,43%
- Conflictos actuales en categorias objetivo: 715
- Conflictos reducibles estimados: 540
- Grupos riesgosos excluidos: 639
- Pares riesgosos detectados y no aplicados: 5.480
- Falsos positivos estimados aplicables: 0

Categorias Mas Beneficiadas

- Mascotas: +73,28%
- Limpieza: +63,64%
- Bebe: +59,17%
- Higiene Personal: +59,13%
- Bebidas: +40,60%

Archivos Creados

- app/fase5a_rules.py
- app/scripts/simular_mejora_matching_fase5a.py
- tests/test_fase5a_rules.py
- tests/test_fase5a_simulacion.py
- reports/fase5a_simulacion.md
- reports/fase5a_detalle_categoria.csv
- reports/fase5a_falsos_positivos.csv
- reports/FASE_5A_REPOORTE.md
- reports/FASE_5A_REPOORTE.pdf
- CAMBIOS_FASE_5A.md

Archivos Modificados

- app/normalizacion.py
- tests/fixtures/productos_reales.json

Tests

Se agregaron pruebas positivas, negativas y casos limite para:

- Bebidas
- Limpieza
- Higiene Personal
- Bebe
- Mascotas
- Simulacion DRY RUN

Resultado final:

- 32 passed

Riesgos Pendientes

- La simulacion excluye 639 grupos riesgosos; esos grupos requieren revision manual.
- Los 5.480 pares riesgosos detectados no deben aplicarse automaticamente.

- La mejora proyectada depende de aplicar solo grupos seguros.
- Las categorías excluidas siguen pendientes para fases posteriores.

Recomendacion Fase 5B

Ejecutar limpieza controlada por categoría, empezando por Mascotas y Limpieza porque muestran mejor retor

Fase 5B

Fecha: 2026-05-31

Objetivo

Aplicar mejoras reales y controladas de matching unicamente en las categorías de menor riesgo:

- Limpieza
- Mascotas

La fase no modifiko Bebidas, Bebe, Higiene Personal, Frutas y Verduras, Congelados, Panaderia, Carnes y P

Cambios Aplicados

- Se creo backup validado antes de modificar la base.
- Se seleccionaron solo grupos seguros usando reglas de Fase 5A.
- Se excluyeron grupos riesgosos, ambiguos o con conflictos de formato, aroma, etapa, sabor o especie.
- Se aplicaron cambios por lote sobre producto_base.
- Se genero trazabilidad completa en CSV.
- Se creo script de rollback.
- Se extendio /diagnostico/matching con metricas de equivalencias actuales, equivalencias por categoría,

Metricas Antes/Despues

- Productos modificados: 502
- Equivalencias en categorías objetivo: 698 -> 924
- Conflictos en categorías objetivo: 234 -> 163

Por categoría:

- Limpieza: 306 productos modificados, equivalencias 451 -> 632, conflictos 166 -> 133.
- Mascotas: 196 productos modificados, equivalencias 247 -> 292, conflictos 68 -> 30.

Seguridad de Datos

Backup pre-aplicacion validado:

- backups/supercheck_pre_fase5b_20260531_215846.db

Rollback disponible:

“powershell

```
python -m app.scripts.rollback_fase5b
""
```

CSV de trazabilidad:

- reports/fase5b_cambios.csv

Validacion:

- reports/fase5b_validacion.md

Archivos Creados

- app/fase5b_apply.py
- app/scripts/aplicar_matching_fase5b.py
- app/scripts/rollback_fase5b.py
- tests/test_fase5b_apply.py
- reports/fase5b_cambios.csv
- reports/fase5b_validacion.md
- reports/FASE_5B_REPORTE.md
- reports/FASE_5B_REPORTE.pdf
- CAMBIOS_FASE_5B.md

Archivos Modificados

- app/matching_diagnostics.py
- reports/AHORRAGO_MASTER_REPORT.md
- reports/AHORRAGO_MASTER_REPORT.pdf
- supercheck.db

Tests

Resultado final esperado de validacion:

- 40 passed
- compileall OK
- Import de app.main OK
- Aplicacion Fase 5B idempotente sin cambios pendientes despues de la primera ejecucion real

Riesgos Pendientes

- Bebidas, Bebe e Higiene Personal quedaron fuera de la aplicacion real.
- Los grupos excluidos por seguridad requieren revision manual o reglas adicionales.
- La base SQLite fue modificada de forma controlada; el rollback esta disponible si se necesita revertir.
- Las categorias de alto riesgo siguen pendientes para fases especificas.

Recomendacion Fase 5C

Preparar una aplicacion controlada para Bebidas usando lista blanca, validacion manual por muestra y regl

Fase 5B-FIX

Fecha: 2026-06-01

Objetivo

Corregir de forma quirurgica 25 productos tipo fideo/pasta afectados por Fase 5B.

Alcance

- No se ejecuto rollback completo de Fase 5B.
- No se modifiko Mascotas.
- No se toco frontend, usuarios, scraping ni migracion de base.
- Se modificaron solo los 25 IDs confirmados por reports/fase5b_cambios.csv.

Resultado

- Productos corregidos en esta ejecucion: 25.
- Productos ya corregidos/idempotentes: 0.
- IDs objetivo restantes en Limpieza > Blanqueadores: 0.
- Fideos de Fase 5B-FIX en Despensa > Fideos: 25.
- Fideos residuales globales en Limpieza > Blanqueadores fuera del alcance Fase 5B-FIX: 14.
- Backup previo: C:\Users\Gonzalo\Pictures\supersuper\backups\supercheck_pre_fix_fideos_fase5b_20260601_2
- CSV de trazabilidad: C:\Users\Gonzalo\Pictures\supersuper\reports\fix_fideos_fase5b.csv.

Rollback Especifico

```
“powershell  
python -m app.scripts.rollback_fix_fideos_fase5b  
“
```

IDs Corregidos

386, 387, 390, 391, 392, 404, 3475, 3479, 3483, 3484, 3485, 3486, 3506, 3507, 3514, 3516, 3518, 3521, 352

Validacion Esperada

- 25 productos quedan en Despensa > Fideos.
- producto_base vuelve al valor anterior registrado en Fase 5B.
- Mascotas no cambia.
- Productos validos de Limpieza no cambian.

Auditoria de Categorias

- Auditoria read-only ejecutada: python -m app.scripts.auditoria_categorias.
- Hallazgos totales: 95.
- mascota_en_higiene: 53.
- alimento_en_limpieza: 31.
- bebida_en_mascotas: 11.
- Los 14 fideos residuales en Limpieza > Blanqueadores no fueron modificados porque no pertenecen a los 2

Validacion Final

- python -m pytest -q: 43 passed.
- python -m compileall app tests: OK.
- python -c "from app.main import app; print(app.title)": FastAPI.

Fase 5D-FIX

Fecha: 2026-06-01

Objetivo

Aplicar correcciones reales de categoria detectadas en Fase 5D, sin tocar falsos positivos.

Resultado

- Productos corregidos en esta ejecucion: 64.
- Productos ya corregidos/idempotentes: 0.
- Backup previo: C:\Users\Gonzalo\Pictures\supersuper\backups\supercheck_pre_fase5d_fix_20260601_203003.d
- CSV de trazabilidad: C:\Users\Gonzalo\Pictures\supersuper\reports\fase5d_fix_cambios.csv.
- Hallazgos restantes post-auditoria: 0.

Detalle por Tipo

- alimento_en_limpieza: 20.
- mascota_en_higiene: 44.
- bebida_en_mascotas: 0.

Seguridad

- No se ejecuto rollback completo.
- No se modificaron falsos positivos.
- No se modificaron bebidas en Mascotas.
- No se toco frontend, usuarios, scraping ni migracion de base.
- producto_base se mantuvo sin recalcular.

Rollback Especifico

```
“powershell
python -m app.scripts.rollback_fix_categorias_fase5d
“
```

Validacion Final

- python -m app.scripts.aplicar_fix_categorias_fase5d: idempotente, 0 cambios nuevos y 64 ya corregidos.
- python -m app.scripts.auditoria_categorias: 0 hallazgos.
- python -m pytest -q: 46 passed.
- python -m compileall app tests: OK.
- python -c "from app.main import app; print(app.title)": FastAPI.

Fase 5C Reporte - AhorraGo

Resumen Ejecutivo

Fase 5C aplico matching real y controlado solo en categorias autorizadas.

Metricas Antes/Despues

- Productos modificados: 22
- Equivalencias: 2218 -> 2225
- Equivalencias ganadas: 7
- Conflictos: 829 -> 829
- Conflictos reducidos: 0

Productos Modificados por Categoria

- Bebe: (0)
- Bebidas: (22)
- Higiene Personal: (0)

Seguridad

- Backup generado: C:\Users\Gonzalo\Pictures\supersuper\backups\supercheck_pre_fase5c_20260601_212826.db
- Rollback disponible: python -m app.scripts.rollback_fase5c
- Auditoria post-cambio: 0 hallazgos
- No se modificaron categorias bloqueadas.

Riesgos

- La fase fue conservadora: Bebidas tuvo cambios; Higiene Personal y Bebe quedaron sin cambios por falta
- Las categorias pendientes requieren reglas de marca/variedad mas completas antes de una aplicacion real

Recomendaciones

- Antes de ampliar Fase 5C, enriquecer MARCAS_CONOCIDAS para bebidas isotonicas/energeticas y reglas de t
- Mantener auditoria de categorias en 0 antes de cualquier nueva aplicacion.

Fase 5F

Fecha: 2026-06-01

Objetivo

Detectar y clasificar errores masivos de categoria/subcategoria antes de usuarios, sin modificar datos.

Modo

- READ ONLY.
- No se modifiko base de datos.
- No se modifiko producto_base.
- No se modificaron categorias ni subcategorias.
- No se toco frontend, usuarios ni scraping.

Resultado

- Hallazgos totales: 1986.
- Alta confianza: 1639.
- Media confianza: 347.
- Baja confianza: 0.
- Falso positivo probable: 0.

Categorías Mas Afectadas

- Bebe: 627
- Desayuno y Snacks: 407
- Lácteos, Huevos y Congelados: 391
- Carnes y Pescados: 213
- Congelados: 104
- Despensa: 99
- Bebidas: 75
- Mascotas: 44
- Frutas y Verduras: 17
- Limpieza: 3

Motivos Principales

- Producto de mascotas fuera de Mascotas: 657
- Producto de higiene personal fuera de Higiene Personal: 457
- Bebida detectada dentro de categoría Bebe: 416
- Bebida fuera de Bebidas: 298
- Snack/fruto seco detectado dentro de categoría Bebe: 126
- Producto de limpieza fuera de Limpieza: 27
- Producto de limpieza detectado dentro de categoría Bebe: 5

Productos Críticos Alta Confianza

ID	Producto	Actual	Sugerida	Motivo
---	---	---	---	---
56	Bebida Leche Láctea Yogu Yogu Chirimoya Caja	Bebe	> Alimentos Bebe	Bebidas > Bebidas Bebida
132	Bebida Leche Láctea Yogu Yogu Mora Caja	Bebe	> Alimentos Bebe	Bebidas > Bebidas Bebida dete
148	Bebida Leche Láctea Yogu Yogu Damasco Caja	Bebe	> Alimentos Bebe	Bebidas > Bebidas Bebida d
149	Bebida Leche Láctea Yogu Yogu Frutilla Caja	Bebe	> Alimentos Bebe	Bebidas > Bebidas Bebida
150	Bebida Leche Láctea Yogu Yogu Pina Caja	Bebe	> Alimentos Bebe	Bebidas > Bebidas Bebida dete
151	Bebida Leche Láctea Trencito Chocolate Caja	Bebe	> Alimentos Bebe	Bebidas > Bebidas Bebida
199	Bebida Vegetal Notmilk Protein Con 7gr Proteína 750 ml NotCo	Bebe	> Alimentos Bebe	Bebidas >
213	Bebida Láctea Sin Lactosa Chocolate 200 ml Milo	Bebe	> Alimentos Bebe	Bebidas > Bebidas Beb
217	Bebida Vegetal Notmilk Chocolate Tetra Pak 200 ml NotCo	Bebe	> Alimentos Bebe	Bebidas > Bebid
218	Bebida Vegetal Notmilk Original 1 L NotCo	Bebe	> Alimentos Bebe	Bebidas > Bebidas Bebida de
220	Bebida Láctea Sabor Frutilla 200 ml Surlat	Bebe	> Alimentos Bebe	Bebidas > Bebidas Bebida d
221	Bebida Láctea Sabor Chocolate 200 ml Surlat	Bebe	> Alimentos Bebe	Bebidas > Bebidas Bebida
222	Bebida Láctea Sabor Vainilla 200 ml Surlat	Bebe	> Alimentos Bebe	Bebidas > Bebidas Bebida d
225	Bebida Vegetal Notmilk Zero 1 L NotCo	Bebe	> Alimentos Bebe	Bebidas > Bebidas Bebida detect
226	Bebida Vegetal Notmilk Low Fat 1 L NotCo	Bebe	> Alimentos Bebe	Bebidas > Bebidas Bebida det
228	Bebida Vegetal Original 1 L Nature's Heart	Bebe	> Alimentos Bebe	Bebidas > Bebidas Bebida d
246	Chocolate Lenguas De Gato Leche 120 g Costa	Lácteos, Huevos y Congelados	> Leche	Mascotas > A

| 286 | Galletas Cachorro Raza Grande Sabor Leche Bolsa 500 g Master Dog | Desayuno y Snacks > Snacks | M
| 300 | Alimento Seco Cachorro Raza Pequena Carne Y Leche Bolsa 3 Kg Master Dog | Carnes y Pescados > Car
| 306 | Alimento Seco Cachorro Raza Mediana/grande Sabor Carne Y Leche Bolsa 3 Kg Master Dog | Carnes y P
| 310 | Galleta Perro Cachoro Leche g g Pet Food | Desayuno y Snacks > Snacks | Mascotas > Alimento Perro
| 339 | Tónico Rose Care Leche & Tónico Micelar 2 En 1 200 ml Nivea | Lacteos, Huevos y Congelados > Lech
| 429 | Bebida Vegetal Notmilk Chocolate 1 L NotCo | Bebe > Alimentos Bebe | Bebidas > Bebidas | Bebida d
| 444 | Bebida Vegetal De Avellana Sabor Chocolate Caja 1 L Vivicosi | Bebe > Alimentos Bebe | Bebidas >
| 509 | Desodorante Spray Nivea Black&white Original Masculino XI Hombre | Lacteos, Huevos y Congelados >

Archivos Generados

- reports/fase5f_clasificacion_masiva.csv
- reports/fase5f_clasificacion_masiva.md
- reports/FASE_5F_REPORTE.md
- reports/FASE_5F_REPORTE.pdf
- CAMBIOS_FASE_5F.md

Recomendacion Fase 5F-FIX

- Crear una fase separada con backup y rollback especifico.
- Aplicar primero solo alta confianza y categorias de destino con subcategoria clara.
- Revisar manualmente hallazgos de media confianza.
- No recalculan producto_base hasta terminar movimientos de categoria.
- Prioridad inicial: Bebidas/Snacks/Limpieza dentro de Bebe y productos de mascotas fuera de Mascotas.

Fase 5G - Reload Test Paralelo

Fecha: 2026-06-01

Objetivo

Crear una BD paralela desde cero y comparar su calidad contra la BD actual sin modificar supercheck.db.

Resultado

- Productos en BD actual: 31124.
- Productos en BD reload: 31124.
- Precios en BD actual: 31139.
- Precios en BD reload: 31155.
- Hallazgos clasificacion masiva actual: 1986.
- Hallazgos clasificacion masiva reload: 1986.
- Hallazgos alta confianza actual: 1639.
- Hallazgos alta confianza reload: 1683.
- Conflictos actual: 1545.
- Conflictos reload: 1605.

Decision

- La BD recargada no tiene menos errores que la actual.
- Los 1986 errores reaparecen en la reload.

- El problema apunta a datos fuente/importadores/reglas de clasificacion.
- No conviene reemplazar supercheck.db por supercheck_reload_test.db.
- Conviene arreglar importadores/datos fuente y seguir con Fase 5F-FIX quirurgica.

Archivos

- reports/FASE_5G_RELOAD_TEST.md
- reports/FASE_5G_RELOAD_TEST.pdf
- reports/comparacion_actual_vs_reload.csv
- reports/auditoria_reload_test.md
- reports/reload_test/

Fase 5H - Root Cause Clasificacion Masiva

Fecha: 2026-06-01

Objetivo

Identificar la causa raiz de los errores masivos que reaparecen despues de una recarga limpia.

Resultado

- Productos trazados: 51.
- Causa raiz principal: scraper_categoria_por_busqueda_amplia.
- Trazas con esa causa: 51 de 51.
- Scripts con evidencia directa: app/scraper_lider.py y app/scraper_jumbo_real.py.
- Scripts con riesgo estructural: app/scraper_unimarc.py, app/combinar_supermercados.py, app/importar_csv

Evidencia

- NotMilk, Yogu Yogu, Milo y Coca-Cola aparecen en Bebe > Alimentos Bebe.
- Master Dog y Pet Food aparecen en Carnes, Desayuno y Snacks o Lacteos.
- Nivea micelar y desodorantes aparecen en Lacteos.
- La categoria erronea ya existe en los CSV fuente trazados.

Decision

- No conviene corregir solo la BD sin arreglar el pipeline.
- No conviene reemplazar la BD actual por reload.
- Conviene ejecutar Fase 5I para corregir scrapers/importadores y agregar validadores pre-import.

Archivos

- reports/FASE_5H_ROOT_CAUSE.md
- reports/FASE_5H_ROOT_CAUSE.pdf
- reports/fase5h_trazabilidad_productos.csv
- reports/fase5h_causa_raiz_resumen.csv

Fase 5I - Pipeline Hardening

Fecha: 2026-06-01

Objetivo

Impedir que categorias imposibles entren al pipeline antes de una nueva recarga.

Cambios

- Se creo app/category_validator.py.
- Se integro la validacion en app/scrapper_lider.py.
- Se integro la validacion en app/scrapper_jumbo_real.py.
- Se integro la validacion en app/scrapper_unimarc.py.
- Se integro la validacion en app/combinar_supermercados.py.
- Se integro la validacion en app/importar_csv.py.
- Se agregaron tests para NotMilk, Coca-Cola, Yogu Yogu, Master Dog, Champion Dog, Pedigree, Nivea, Rexon

Resultado Estimado

- Hallazgos Fase 5F: 1986.
- Bloqueados por validador: 1934.
- Remanente estimado: 52.
- Objetivo solicitado: menos de 300 hallazgos.

Seguridad

- No se modifiko supercheck.db.
- No se ejecuto recarga.
- No se ejecuto scraping.

Siguiente Paso

Ejecutar Fase 5J: reload test post-hardening en BD paralela y revision de rechazos.

Archivos

- reports/FASE_5I_PIPELINE_HARDENING.md
- reports/FASE_5I_PIPELINE_HARDENING.pdf
- CAMBIOS_FASE_5I.md

Fase 5J-R Fix del Validador de Categorias

Fecha: 2026-06-02

Objetivo

Corregir el validador antes del reload para no convertir falsos positivos en borrados permanentes.

Resultado

- Borrados a la fuerza: 1.440 (v1) -> 999 (v2, solo confianza Alta).
- Productos rescatados de borrado: 441 (287 a cuarentena + 154 falsos positivos eliminados).
- Falsos positivos corregidos: vinagre de vino, Maggi Jugoso, carne al vino, bebida lactea.
- Aciertos del bug 5H conservados: Pedigree/Whiskas en Carnes, Nivea/Cif en Crema.
- Tests: 16/16 PASSED.

Cambios

- Coincidencia por palabra completa (regex) en vez de substring.
- Exclusiones de bebidas (vinagre, en/al vino, bebida lactea, saborizante, polvo, salsa).
- Politica de cuarentena: Alta=reject, Media=conservar+revisar.
- Categoria insensible a acentos.

Archivos

- app/category_validator.py
- tests/test_category_validator.py
- reports/FASE_5JR_FIX_VALIDADOR.md / .pdf
- reports/v2_reject_alta.csv, reports/v2_quarantine_media.csv